1. INTRODUCTION

1.1 About the Project

The IFSC Code Finder is a web-based application designed to simplify the process of retrieving Indian Financial System Codes (IFSC) for bank branches across India. The IFSC code is a unique alphanumeric identifier assigned by the Reserve Bank of India (RBI) to each bank branch to facilitate secure and efficient electronic fund transfers, including NEFT (National Electronic Funds Transfer), RTGS (Real-Time Gross Settlement), and IMPS (Immediate Payment Service) transactions. This project aims to provide a fast, reliable, and user-friendly platform where users can quickly search for an IFSC code using various parameters such as bank name, state, district, and branch name**.** Traditionally, individuals and businesses had to rely on printed documents, bank websites, or customer service inquiries to obtain IFSC codes, which was often time-consuming and inefficient. This web application eliminates these challenges by offering an automated, real-time search tool that ensures users get accurate results within seconds.

The backend of the system is built using PHP and MySQL, ensuring a robust and efficient database management system that stores a large number of IFSC codes and associated bank details. The frontend, developed using HTML, CSS, and JavaScript, provides a smooth and intuitive interface, making it easy for both technical and non-technical users to access the required information. The system is designed to be responsive and accessible across multiple devices, including desktops, laptops, tablets, and smartphones. To enhance usability, the application incorporates search filters and dropdown menus, allowing users to refine their searches efficiently. The search algorithm is optimized to deliver quick and precise results, reducing errors and improving the accuracy of financial transactions.

Security is a key aspect of the project, with preventive measures in place to protect against unauthorized access and data breaches. The modular structure of the application ensures that updates and modifications can be made easily, especially as banks introduce new branches or update existing IFSC codes. This project is highly beneficial for bank customers, businesses**,** financial institutions, and developers who require quick access to IFSC codes for seamless online banking transactions. Furthermore, the system can be integrated with the other banking

applications or financial management tools, enhancing its versatility. The real-time data retrieval process ensures that users receive the most up-to-date information, eliminating discrepancies in transactions.

By automating the IFSC code search process, this project significantly reduces the chances of incorrect entries, thereby minimizing the risk of failed transactions or financial discrepancies**.** The system is designed to be scalable and can accommodate future expansions, making it an essential tool for individuals and businesses involved in frequent digital transactions. The IFSC Code Finder is not just a simple search tool but a powerful web application that enhances the security, accuracy, and efficiency of online banking operations. Its ease of use, rapid search capabilities, and comprehensive database management system make it a valuable solution for modern digital banking needs.

2. SYSTEM SPECIFICATION

2.1 Hardware Specification

COMPONENT	SPECIFICATION
Processor	Pentium-IV
Speed	1.1 GHZ
Ram	512 GB
Hard Disk	40 GB
Key Board	Standard Window Key
Mouse	SVGA

2.2 Software Specification

COMPONENT	SPECIFICATON
Operating System	Windows 8/7/10
Tools	XAMPP, VS Code
Front End	HTML, CSS, JavaScript, Bootstrap
Back End	PHP, MySQL, Apache

2.2.1 Software Description

Introduction To Front End

Html

HTML (Hypertext Markup Language) is the standard language used to create and structure webpages. It consists of a series of elements (tags) that define different parts of a webpage, such as headings, paragraphs, images, links, tables, and forms. HTML works as the foundation of any website, providing the content and structure that browsers interpret and display. Each HTML document starts with a <!DOCTYPE html> declaration, followed by the <html> tag that contains <head> (for metadata, styles, and scripts) and <body> (for visible content). HTML elements are enclosed in angle brackets (< >), and most have an opening and closing tag, such as for paragraphs (...). Some tags, like <image> and <break>, are self-closing.HTML allows for the integration of CSS (for styling) and JavaScript (for interactivity) to create fully functional and visually appealing websites.

Css

CSS (Cascading Style Sheets) is a stylesheet language used to control the appearance and layout of HTML elements on a webpage. It allows developers to define styles such as colors, fonts, margins, padding, backgrounds, and positioning to enhance the visual appeal of a website. CSS follows a cascade rule, meaning styles can be applied inline (style attribute), internally (<style> tag in the <head>), or externally (linked .CSS file). It also supports selectors (e.g., element, class, ID) and advanced features like flexbox, grid, animations, and media queries for responsive design. By separating content (HTML) from presentation (CSS), it ensures cleaner, more maintainable code and a better user experience

Javascript

JavaScript is a powerful, lightweight programming language used to add interactivity and dynamic behavior to webpages. It enables real-time updates, animations, event handling, and form validation without needing to reload the page. JavaScript runs in the browser and interacts with the Document Object Model (DOM) to manipulate HTML and CSS elements. It supports variables, functions, loops, and conditional statements for logical operations. JavaScript works with APIs to fetch and process data, enabling web applications like chatbots, calculators, and dynamic content rendering. Modern frameworks like React, Angular, and Vue.js enhance its capabilities. It also supports asynchronous programming with promises and async or wait for handling API requests efficiently. JavaScript can be used both on the client-side and server-side (e.g., Node.js). Its versatility makes it essential for web development.

Bootstrap

Bootstrap is a popular front-end framework used for designing responsive and mobile-friendly websites quickly. It provides a collection of pre-built CSS styles, JavaScript components, and a grid system that simplifies web development. Bootstrap includes ready-to-use elements like buttons, forms, modals, alerts, carousels, and navigation bars, reducing the need for writing extensive custom CSS and JavaScript. Its 12-column responsive grid system allows developers to create flexible layouts that adapt to different screen sizes. Bootstrap also supports utility classes, making styling easier without writing custom CSS. With built-in themes and icons, it ensures consistency across designs. Being open-source, it is widely used for developing both simple websites and complex web applications efficiently.

Introduction To Back End

Php

PHP (Hypertext Preprocessor) is a server-side scripting language widely used for web development. It is embedded in HTML and executed on the server, generating dynamic web pages before it are sent to the client's browser. PHP is commonly used to handle form data, manage sessions, interact with databases (like MySQL), and create user authentication systems**.** It supports various programming paradigms, including procedural, object-oriented, and functional programming. PHP allows easy integration with HTML, CSS, JavaScript, and databases, making it a powerful choice for building websites and applications. With built-in functions for file handling, encryption, and API communication, PHP simplifies web development. It is widely used in CMS platforms like WordPress, Drupal, and Joomla. PHP supports frameworks like Laravel and CodeIgniter for scalable development. As an open-source language, it is constantly updated and supported by a large developer community.

MYSQL

MySQL is an open-source Relational Database Management System (RDBMS) used to store, manage, and retrieve structured data efficiently. It organizes data into tables with rows and columns, allowing for easy access using SQL (Structured Query Language)**.** MySQL is widely used in web applications to handle user data, transactions, and content management systems like WordPress, Joomla, and Drupal. It supports CRUD operations (Create, Read, Update, Delete) and advanced features like indexing, joins, stored procedures, and triggers. MySQL works well with programming languages like PHP, Python, and Java**,** making it ideal for dynamic web applications. It ensures data integrity, security, and scalability, supporting large-scale applications. MySQL can be used in cloud and on-premise environments and is known for its speed and reliability. It is commonly used in e-commerce, finance, and social media applications for efficient data management.

3. SYSYEM STUDY

3.1 Existing System

The existing system for finding IFSC codes relies on multiple manual and online sources, which can be time-consuming and inefficient. Traditionally, users had to refer to bank-provided documents, printed directories, or customer service inquiries to obtain the correct IFSC code for a specific branch. Many banks provide IFSC codes on their official websites, but users must navigate through complex menus and search for the correct details, which is not always user-friendly. Additionally, third-party websites and financial portals offer IFSC search tools, but it may contain outdated or incorrect information, leading to potential errors in transactions. Another major limitation of the existing system is the lack of a centralized, real-time search tool that provides quick and accurate results. Many websites and applications require users to manually enter branch details without any smart filtering or search suggestions, increasing the chances of incorrect inputs. Moreover, security concerns arise when using unverified third-party websites that may store or misuse user data. The absence of an efficient, automated, and up-to-date database system also results in difficulties for frequent users such as bank employees, financial institutions, and businesses that require IFSC codes regularly for online transactions. Additionally, existing methods do not provide a mobile-friendly or responsive design, making it difficult for users to access information on different devices. Given these challenges, a more streamlined, secure, and automated solution is required to **s**implify the IFSC code search process and ensure accuracy in financial transactions**.**

3.1.1 Drawbacks of Existing System

Time-Consuming Search Process – Users often have to manually search through bank websites, printed directories, or third-party platforms, which can be slow and cumbersome.

Lack of a Centralized Database – IFSC codes are scattered across multiple sources, making it difficult to find accurate and updated information in one place.

Outdated or Incorrect Information – Many third-party websites do not update their databases regularly, leading to incorrect IFSC codes and failed transactions.

No Smart Search Features – The existing systems often require users to enter full details manually without providing smart suggestions or filters to simplify the process.

Complex Navigation – Many bank websites have complicated interfaces, making it difficult for non-technical users to locate IFSC codes quickly.

Security Risks – Third-party websites may store, misuse, or leak sensitive banking information**,** leading to potential security threats.

Dependency on Bank Customer Support – In some cases, users need to contact bank representatives or visit branches to get IFSC codes, which is inconvenient.

No Mobile-Friendly Design – Many existing platforms are not optimized for mobile devices, making it harder for users to search on smartphones or tablets.

Lack of Real-Time Updates – Changes in IFSC codes due to bank mergers or branch relocations may not be reflected in outdated databases.

Increased Risk of Transaction Failures – Using an incorrect or outdated IFSC code can result in delayed or failed financial transactions.

3.2 Proposed System

The proposed system, IFSC Code Finder, is a web-based application designed to provide a fast, accurate, and user-friendly solution for retrieving Indian Financial System Codes (IFSC) for bank branches across India. This system eliminates the inefficiencies of the existing methods by offering a centralized database where users can quickly search for IFSC codes using multiple filters such as bank name, state, district, and branch name. Unlike traditional methods that rely on manual searches or third-party websites with outdated information**,** this application ensures real-time data retrieval and maintains an up-to-date database of all bank branches. The proposed system is built using PHP and MySQL for the backend, which efficiently manages search queries and ensures secure database handling. The frontend, developed with HTML, CSS, and JavaScript, provides an intuitive and responsive interface, making it accessible across desktops, tablets, and mobile devices. To enhance search efficiency, the system incorporates smart filtering and auto-suggestion features, reducing user input errors and improving the accuracy of searches. Additionally, AJAX and jQuery can be integrated to enable real-time search functionality without reloading the page. Security is a major focus of the proposed system, with proper validation, encryption, and access control mechanisms to prevent unauthorized access and data breaches. Unlike existing systems that require users to navigate complex bank websites or contact customer support, this solution provides a one-click, automated search process for retrieving IFSC codes instantly. Furthermore, the system's modular structure allows for future updates, making it scalable as banks introduce new branches or update existing IFSC codes. By implementing this IFSC Code Finder, users can experience a faster, more secure, and error-free method of obtaining IFSC codes, reducing transaction failures and improving digital banking efficiency. The proposed system is particularly beneficial for bank customers, businesses, and financial institutions that require frequent access to accurate banking details for seamless online transactions.

3.2.1 Advantages of Proposed System

User-Friendly Interface – A simple and intuitive design makes it easy for users to navigate and search for IFSC codes.

Fast and Accurate Search – Users can quickly find the IFSC code by entering details like bank name, state, district, and branch name.

Smart Filtering and Auto-Suggestions – Helps users refine their search efficiently, reducing errors and improving accuracy.

Real-Time Data Retrieval – Uses a structured database (MySQL) to provide up-to-date IFSC codes for all bank branches.

Responsive Design – The application works seamlessly on desktops, tablets, and mobile devices for better accessibility.

Secure Data Handling – Implements data validation and security measures to prevent unauthorized access and misuse.

Quick Navigation – Users can easily switch between different banks and branches using dropdown menus and search filters.

Backend Management with PHP & MySQL – Ensures efficient database management and query handling for a smooth user experience.

Real-Time Search Using AJAX – Enables searching without reloading the page, improving performance and user experience.

Scalable and Easily Updatable – The modular structure allows for easy updates when banks modify or add new IFSC codes.

Error Handling and Validation – Prevents incorrect or incomplete searches by guiding users with input validation messages.

Cross-Browser Compatibility – Works on different web browsers like Chrome**,** Firefox, Edge, and Safari without issues.

Lightweight and Fast Performance – Optimized to load quickly, even on slow internet connections.

No Need for Manual Lookup – Eliminates the hassle of referring to printed directories or bank websites for IFSC codes.

Beneficial for Businesses and Financial Institutions – Helps companies and banks streamline online transactions with accurate IFSC details.

4. SYSTEM DESIGN AND DEVELOPMENT

4.1 Data Flow Diagram

A data flow diagram is graphical tool used to describe and analyze movement of data through a system. These are the central tool and the basis from which the other components are developed. The transformation of data from input to output, through processed, may be described logically and independently of physical components associated with the system. These are known as the logical data flow diagrams. The physical data flow diagrams show the actual implements and movement of data between people, departments and workstations. A full description of a system actually consists of a set of data flow diagrams. Using two familiar notations Yourdon, Gane and Sarson notation develops the data flow diagrams. Each component in a DFD is labeled with a descriptive name. Process is further identified with a number that will be used for identification purpose. The development of DFD'S is done in several levels. Each process in low level diagrams can be broken down into a more detailed DFD in the next level. The top-level diagram is often called a context diagram.

Context Diagram:

It contains a single process, but it plays a very important role in studying the current system. The context diagram defines the system that will be studied in the sense that it determines the boundaries. Anything that is not inside the process identified in the context diagram will not be part of the system study. It represents the entire software element as a single bubble with input and output data indicated by incoming and outgoing arrows respectively.

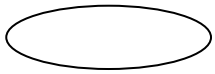
A DFD is also known as a bubble chart has the purpose of clarifying system requirements and identifying major transformations that will become programs in system design. So, it is the starting point of the design to the lowest level of detail. A DFD consists of a series of bubbles joined by data flows in the system.

DFD Symbols:

In the DFD, there are four symbols

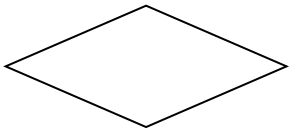
1. A square defines a source(originator) or destination of system data.
2. An arrow identifies data flow. It is the pipeline through which the information flows. Data move in a specific direction from an origin to a destination.
3. A circle or a bubble represents a process that transforms incoming data flow into outgoing data flows.
4. An open rectangle is a data store, data at rest or a temporary repository of data.

Symbols Elementary references



Process Start

Data Flow



Decision

DFD a Constructing:

Several rules of thumb are used in drawing DFD'S:

1. Process should be named and numbered for an easy interface. Each name should be representative of the process.
2. The direction of flow is from top to bottom and from left to right. Data traditionally flow from source to the destination although it may flow back to the source. One way to indicate this is to draw long flow line back to a source. An alternative way is to

repeat the source symbol as a destination. Since it is used more than once in the DFD it is marked with a short diagonal. When a process is exploded into low level details, it gets numbered.

3. The names of data stores and destinations are written in capital letters. Process and dataflow names have the first letter of each work capitalized.
4. A DFD typically shows the minimum contents of data store. Each data store should contain all the data elements that flow in and out.
5. Questionnaires should contain all the data elements that flow in and out. Missing the interfaces redundancies and like is then accounted for often through interviews.

Salient features of DFD'S:

1. The DFD shows flow of data, not of control loops and decision are controlled considerations do not appear on a DFD.
2. The DFD does not indicate the time factor involved in any process whether the data flow take place daily, weekly, monthly or yearly.
3. The sequence of events is not brought out on the DFD.

Types of Data Flow Diagrams

DFD's are of two types

a. Physical DFD

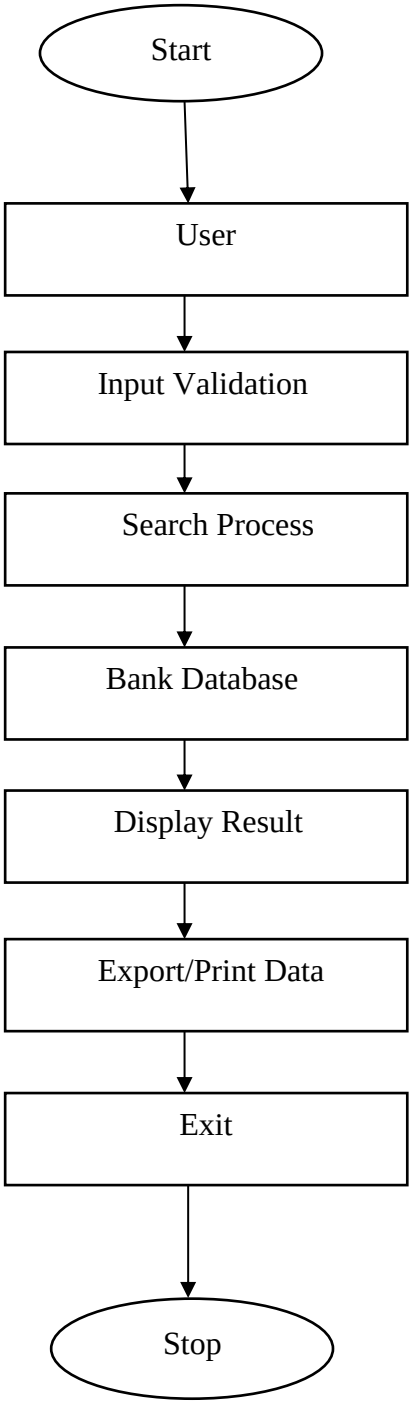
b. Logical DFD

Physical DFD:

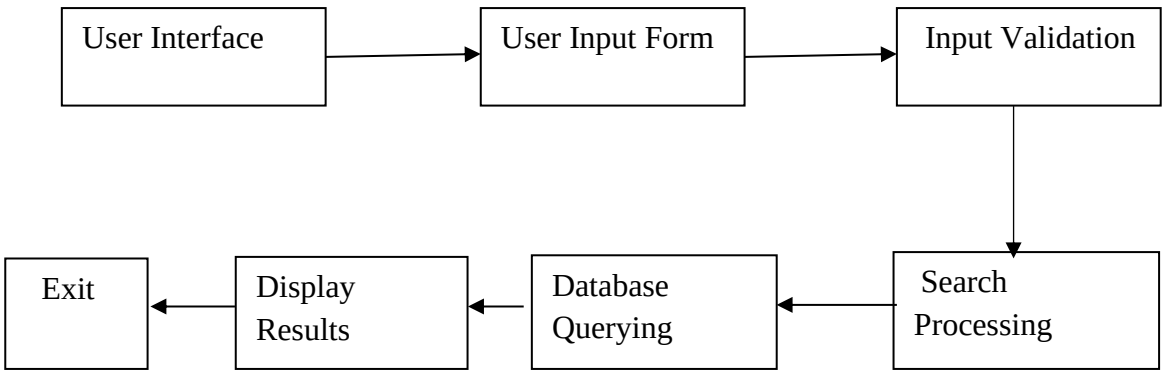
Structured analysis states that the current system should be first understand correctly. The physical DFD is the model of the current system and is used to ensure that the current system has been clearly understood. Physical DFDs shows actual devices, departments, and people etc., involved in the current system

Logical DFD:

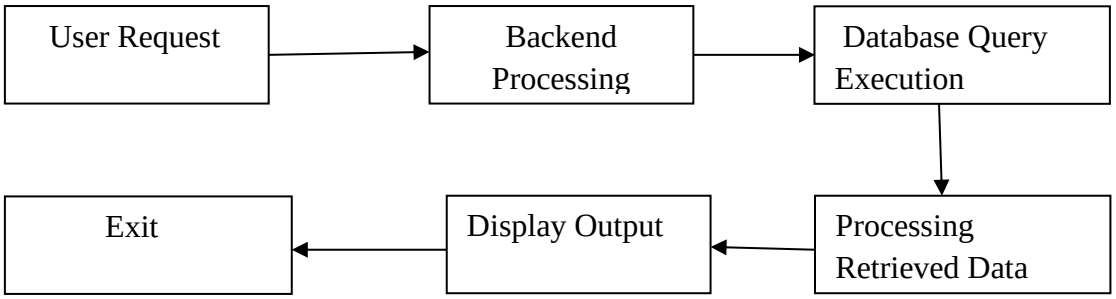
Logical DFDs are the model of the proposed system. This clearly should show the requirements on which the new system should be built. Later during design activity this is taken as the basis for drawing the system's structure charts



4.2 Input Design



4.3 Output Design



4.4 Database Description

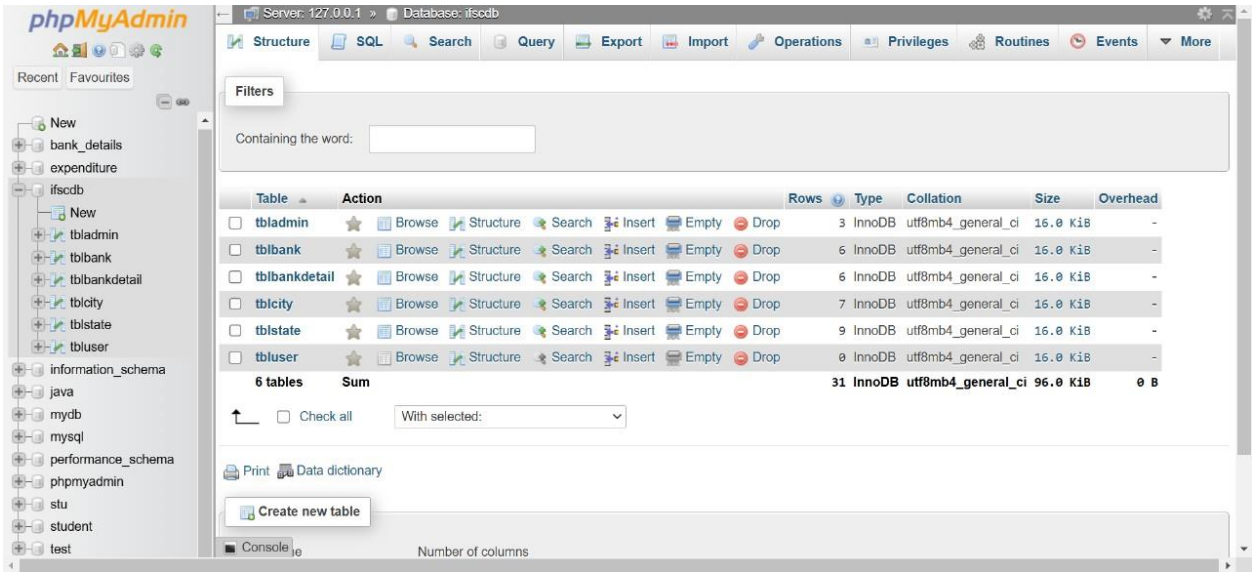
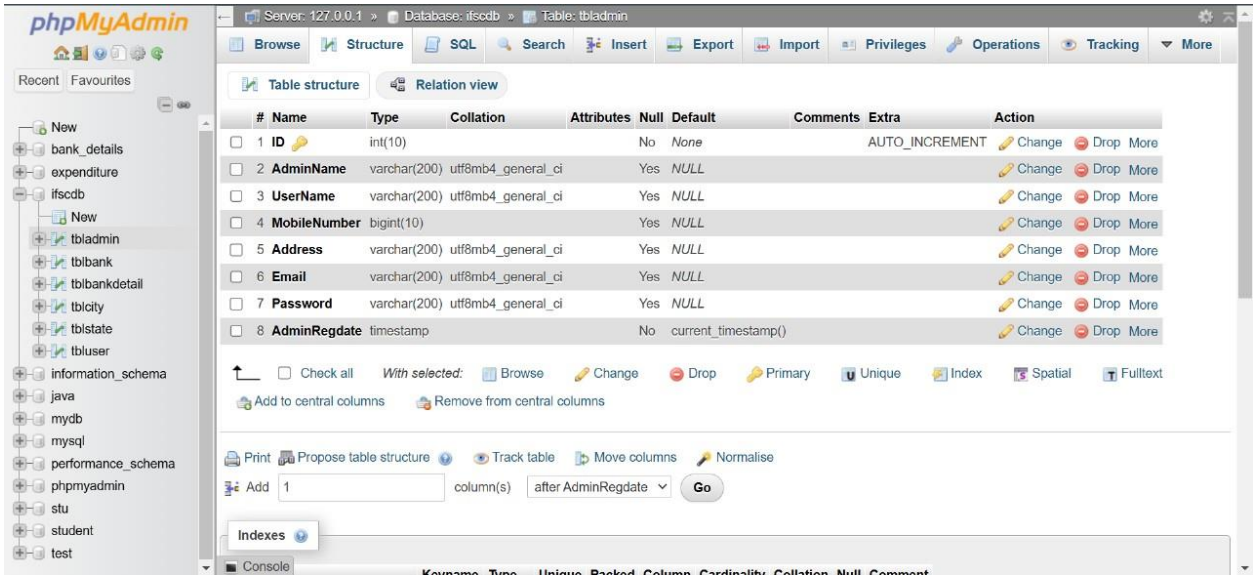


Table Structure



4.5 Module Description

1.**User Authentication Module (Optional)**

This module manages user access, allowing registered users or administrators to log in and perform operations. It ensures secure access to sensitive data and prevents unauthorized modifications**.**

2**. Search Module**

This is the core module where users enter search criteria such as Bank Name, State**,** District, and Branch Name to find the IFSC code. It includes auto-suggestions and smart filtering to enhance user experience and reduce errors.

3. **Input Validation Module**

This module ensures that users enter correct and complete data before processing the request. It checks for missing fields, incorrect spellings, and unsupported values to prevent invalid searches.

4. **Database Management Module**

This module handles all database operations using MySQL**.** It stores and retrieves IFSC codes, bank names, branch details, addresses, and MICR codes, ensuring accurate and up-to-date information.

5. **Search Processing Module**

After user input is validated, this module processes the request using PHP logic to generate a database query. It ensures optimized and fast retrieval of IFSC codes.

6. **Result Display Module**

This module formats and presents the search results in a user-friendly interface. It displays the IFSC Code, Bank Name, Branch, Address, and MICR Code in a structured format.

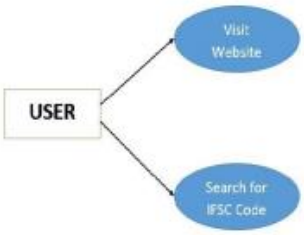
7. **Export & Print Module (Optional)**

Users can print, copy, or export the retrieved IFSC details in different formats such as PDF or CSV, making it easier to share or store for future reference.

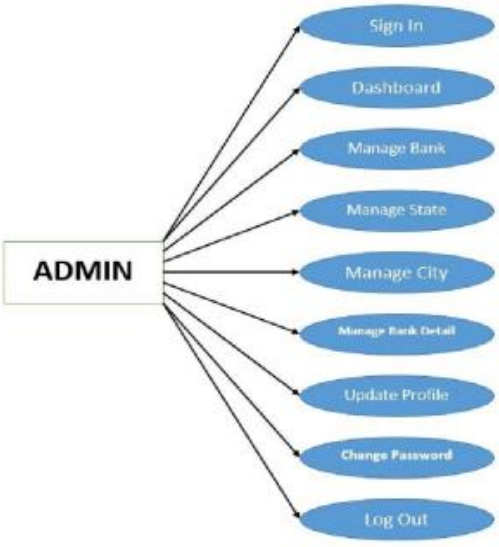
8**. Security Module**

Ensures data protection, access control, and encryption to prevent unauthorized access or modifications. It safeguards the database against SQL injection, cross-site scripting (XSS), and other vulnerabilities**.**

Use Case Diagram for User Module



Use Case Diagram for Admin Module



5. TESTING AND IMPLEMENTATION

5.1 System Testing

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, subassemblies, assemblies and/or a finished product it is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of Test. Each test type addresses a specific testing requirement

5.1.1 Unit Testing

Unit testing is a type of software testing in which individual units (components) of software are tested to verify that it meet their requirements. Unit testing is typically done by developers to ensure that the code have written meets the requirements set out in the design documents. It is important to note that unit testing does not generally include testing of the system as a whole, but rather focuses on the individual units. Unit tests are usually small tests that validate that a particular unit of code works as expected. Unit tests should be independent of each other, so that it can be run in any order without affecting the outcome. Unit tests should also be written in a way that it can be easily maintained and extended. Unit tests are an important part of system testing, as it provide a way to quickly identify and fix errors in the system. By running unit tests as part of system testing, tester can ensure that the system is functioning correctly, and that any issues that arise can be addressed quickly. This helps to reduce the cost of maintenance and repair, and ensures that the system is functioning as expected.

5.1.2 Integration Testing

Integration testing is a type of system testing that is used to test the integration of different components of a system. It is used to test the integration between different software systems and hardware components. Integration testing is performed after unit testing and before system testing. The purpose of integration testing is to verify that different components of the system work together as expected. It is used to detect any errors or issues that may occur when the different components are integrated.

5.1.3 Validation Testing

Validation testing is a type of software testing that is performed to verify that the software meets the customer's functional and technical requirements. It is typically done after the system has been integrated into a larger system and is intended to ensure that the software is performing as expected. Validation testing is usually performed by an independent team of testers, who are familiar with the customer's requirements and the system's architecture. Tester use various test tools and techniques to test the system's functionality and performance, as well as its integration with other systems. The goal of validation testing is to verify that the system meets the customer's requirements, and is able to perform its intended functions. The tests should also identify any potential areas of improvement that can be addressed with additional development or customization. Validation testing is an important step in the software development process, as it ensures that the system meets the needs of the customer and that it is functioning as expected.

5.1.4 Output Testing

System testing is a testing technique used to determine the functionality of a system. It tests the system as a whole and not the individual components or units. Output testing is a type of system testing that verifies the output of the system against the expected results. It is used to check if the system is producing the expected output or not. Output testing is important in system testing as it helps to ensure that the system is working as expected and that any issues that arise during the testing process can be identified and resolved.

5.1.5 Acceptance Testing

Acceptance testing is the process of verifying that a system meets the requirements of the customer. It is typically done in system testing, when the system has been integrated and is ready for external testing. The goal of acceptance testing is to ensure that the system meets the customer’s requirements and can be deployed in production. Acceptance testing may involve testing against user stories, use cases, and/or acceptance criteria. It may also involve usability testing, performance testing, and compatibility testing.

5.2 System Implementation

Implementation is the stage in the project where the theoretical design is turned into a working system. The implementation phase constructs, installs and operates the new system. The most crucial stage in achieving a new successful system is that it will work efficiently and effectively.

There are several activities involved while implementing a new project.

1. End user Training
2. End user Education
3. Training on the application software

All projects go through a life cycle beginning with defining how the new software package will be used in an organization (requirements) through the end point of the project – a successful and effective implementation. Our activities matrix has been organized around six generic implementation life cycle phases.

1. **Business Requirement and Proposed Solution** – this is the phase where the business requirements are finalized, the software package is learned, and a solution using the package is defined to meet the business requirements.
2. **High Level Design (Functional Specifications)**–the planned solution is further clarified by functionally specifying how the system will operate.
3. **System Implementation** – in this phase the system is implemented and operations are converted to the new system.

CONCLUSION AND FUTURE ENHANCEMENT

Conclusion

The IFSC Code Finder project provides a fast, efficient, and user-friendly solution for retrieving Indian Financial System Codes (IFSC) with ease. By integrating PHP, MySQL, HTML, CSS, JavaScript, and AJAX, the system ensures real-time, accurate, and secure data retrieval. Unlike traditional methods that rely on manual searches or outdated third-party sources**,** this web application offers a centralized platform where users can quickly find IFSC codes by simply entering the bank name, state, district, and branch details. The system's smart filtering, auto-suggestions, and real-time search features improve user experience and eliminate input errors. Additionally, security measures such as input validation, encryption, and database protection ensure safe access to sensitive banking information. The responsive design makes the application accessible across multiple devices, enhancing usability for both individual users and businesses. The inclusion of export and print options further adds convenience for users who need to store or share IFSC details for banking transactions. Overall, this project successfully addresses the limitations of the existing system by providing a modern, automated, and scalable solution. It enhances the accuracy, security, and efficiency of IFSC code retrieval, making it a valuable tool for customers, financial institutions, and businesses involved in digital banking transactions. With its structured design and future scalability, the system can be expanded to include more features, ensuring it remains a reliable and effective banking support tool in the long run.

Future Enhancement

Future enhancements for the IFSC Code Finder project include integrating API-based real-time IFSC updates to ensure accuracy and eliminate outdated information. A mobile application can be developed for better accessibility and ease of use. Adding voice search functionality will further improve user experience. Enhanced AI-based predictive search can make searching faster and more efficient. Additionally, multi-language support can help reach a wider audience across India.

BIBLIOGRAPHY

Referred Books

1. HTML & CSS : The Complete Reference, Fifth Edition by Thomas Powell, 2010.
2. A Smarter Way to Learn JavaScript by Mark Myers, 2014
3. Learning Web Development with Bootstrap and Angular JS by Stephen Radford, 2015
4. Head First PHP & MySQL by Michael Morrison, 2008
5. A beginner's guide to HTML, CSS, JavaScript and Web Graphics by Jennifer Niederst Robbins, 2018

Referred Websites

- Official Bank Websites
- Reserve Bank of India (RBI) Website
- www.w3schools.com
- www.php.net
- www.mysql.com
- www.stackoverflow.com

APPENDICES

A. Source Code

Index.php

```
<?php
Session start ();
//error reporting (0);
include('admin/includes/dbconnection.php');
?>
<!doctype html>
<html class="no-js" lang="en">
<head>
<! --====== Title ======-->
<title>IFSC Code Finder Portal | Home</title>
 <! --====== Slick CSS ======-->
 <link rel="stylesheet" href="assets/css/slick.css">
 <! --====== Font Awesome CSS ======-->
 <link rel="stylesheet" href="assets/css/font-awesome.min.css">
 <! --====== Line Icons CSS ======-->
 <link rel="stylesheet" href="assets/css/LineIcons.css">
  <! --====== Animate CSS ======-->
```

```
<link rel="stylesheet" href="assets/css/animate.css">

<! --====== Magnific Popup CSS ======-->

<link rel="stylesheet" href="assets/css/magnific-popup.css">

<! --====== Bootstrap CSS ======-->

<link rel="stylesheet" href="assets/css/bootstrap.min.css">

<! --====== Default CSS ======-->

<link rel="stylesheet" href="assets/css/default.css">

<! --====== Style CSS ======-->

<link rel="stylesheet" href="assets/css/style.css">

</head>

<body>

<! --[if IE]>

<p class="browser upgrade">You are using an <strong>outdated</strong> browser. Please <a href = "https://browsehappy.com/">upgrade your browser</a> to improve your experience and security. </p>

<! [endif]-->

<! --====== PRELOADER PART START ======-->

<div class="pre loader">

<div class="loader">

<div class="ytp-spinner">

<div class="ytp-spinner-container">
```

```
<div class="ytp-spinner-rotator">

<div class="ytp-spinner-left">

<div class="ytp-spinner-circle"></div>

</div>

<div class="ytp-spinner-right">

<div class="ytp-spinner-circle"></div>

</div>

</div>

</div>

</div>

</div>

</div>

<! -- ====== PRELOADER PART ENDS ======-->

<! -- ====== HEADER PART START ======-->

<header class="header-area">

<div class="navbar-area headroom">

<div class="container">

<div class="row">

<div class="col-lg-12">

<nav class="navbar navbar-expand-lg">
```

<h3 style="color: red; padding-right: 50px;">IFSC Code Finder Portal</h3>

<button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarSupportedContent" aria-controls="navbarSupportedContent" aria-expanded="false" aria-label="Toggle navigation">

</button>

<ul id="nav" class="navbar-nav m-auto">

<li class="nav-item active">

Home

<li class="nav-item">

Admin

</nav> <! -- navbar -->

 <! -- row -->

 <! -- container -->

 <! -- navbar area -->

```
<div id="home" class="header-hero bg_cover d-lg-flex align-items-center"
style="background-image: url(assets/images/header-hero.jpg)">

<div class="container">

<div class="row">

<div class="col-lg-7">

<div class="header-hero-content">

<h1 class="hero-title wow fadeInUp" data-wow-duration="1s" data-wow-
delay="0.2s"><b>Search</b> <span>Bank</span> Detail <b>by one click. </b></h1>

<div class="header-singup wow fadeInUp" data-wow-duration="1s" data-wow-
delay="0.8s">

<form action="search.php" method="post" name="search">

<input type="text" placeholder="Enter Bank Name/Zipcode/Branch"
name="searchifsccode">

<button class="main-btn" name="search" id="submit"
type="submit">Search</button></form>

</div>

</div> <! -- header hero content -->

</div>

</div> <! -- row -->

</div> <! -- container -->

<div class="header-hero-image d-flex align-items-center wow fadeInRightBig" data-
wow-duration="1s" data-wow-delay="1.1s">

<div class="image">
```

<img src="assets/images/hero-image.png" alt="Hero Image">

 <! -- header hero image -->

 <! -- header hero -->

</header>

<! --====== FOOTER PART START ======-->

<footer id="footer" class="footer-area bg_cover" style="background-image: url(assets/images/footer-bg.jpg)">

© <?php echo date('Y');?> IFSC Code finder Portal

 <! -- container -->

</footer>

<! --====== FOOTER PART ENDS ======-->

<! --====== BACK TOP TO PART START ======-->

<! --====== BACK TOP TO PART ENDS ======-->

<! --====== Jquery js ======-->

<script src="assets/js/vendor/jquery-1.12.4.min.js"></script>

<script src="assets/js/vendor/modernizr-3.7.1.min.js"></script>

<! --====== Bootstrap js ======-->

<script src="assets/js/popper.min.js"></script>

<script src="assets/js/bootstrap.min.js"></script>

<! --====== Slick js ======-->

<script src="assets/js/slick.min.js"></script>

<! --====== Isotope js ======-->

<script src="assets/js/imagesloaded.pkgd.min.js"></script>

<script src="assets/js/isotope.pkgd.min.js"></script>

<! --====== Counter Up js ======-->

<script src="assets/js/waypoints.min.js"></script>

<script src="assets/js/jquery.counterup.min.js"></script>

<! --====== Circles js ======-->

<script src="assets/js/circles.min.js"></script>

<! --====== Appear js ======-->

<script src="assets/js/jquery.appear.min.js"></script>

<! --====== WOW js ======-->

<script src="assets/js/wow.min.js"></script>

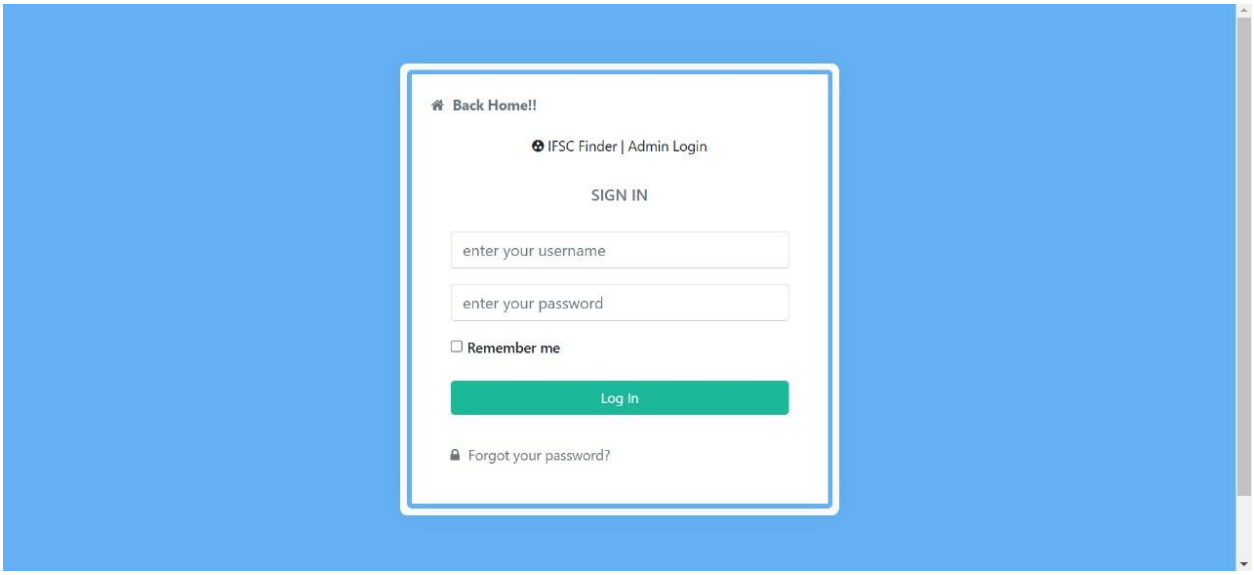
<! --====== Headroom js ======-->

<script src="assets/js/headroom.min.js"></script>

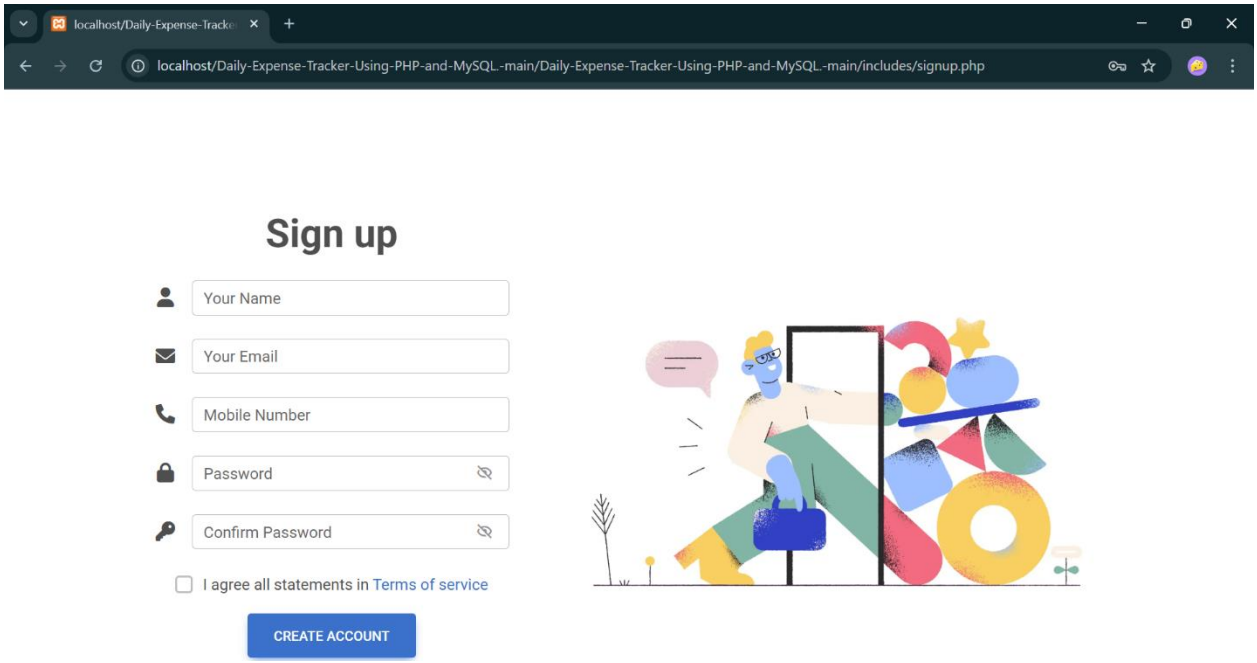
<! --====== Jquery Nav js ======-->

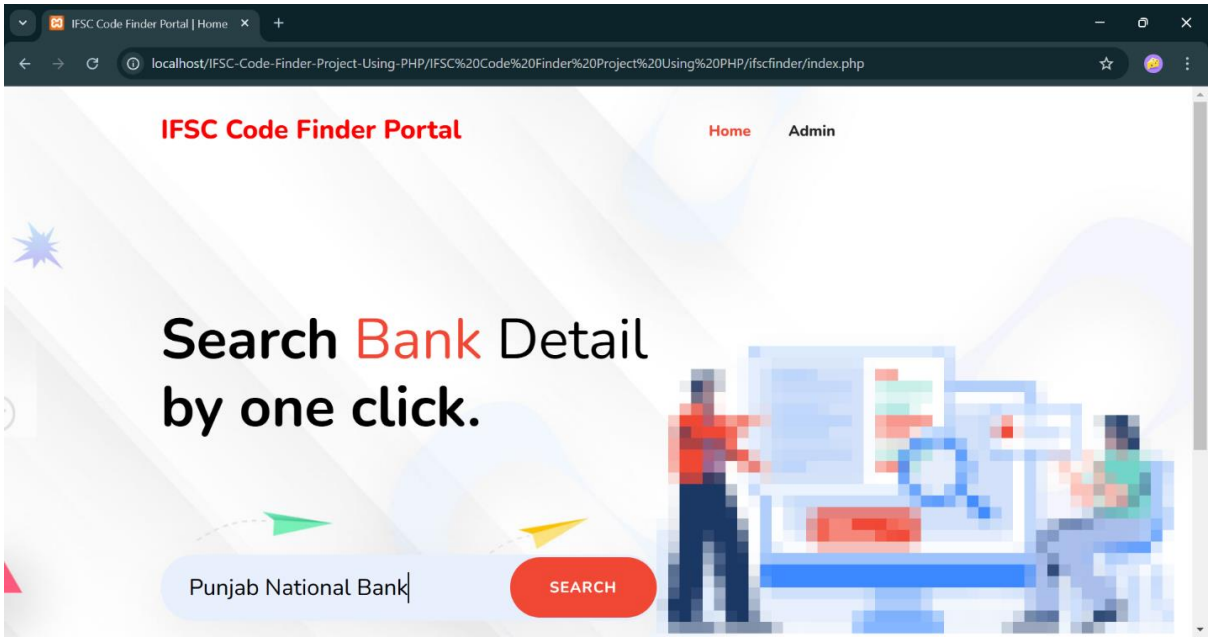
```
<script src="assets/js/jquery.nav.js"></script>

<! --====== Scroll It js ======-->

<script src="assets/js/scrollIt.min.js"></script>

<! --====== Magnific Popup js ======-->

<script src="assets/js/jquery.magnific-popup.min.js"></script>

<! --====== Main js ======-->

<script src="assets/js/main.js"></script>

</body>

</html>
```

B. Sample Input



C. Sample Output




IFSC Code Finder Portal | Home
localhost/IFSC-Code-Finder-Project-Using-PHP/IFSC%20Code%20Finder%20Project%20Using%20PHP/ifscfinder/index.php
IFSC Code Finder Portal
Home
Admin
Search Bank Detail
by one click.
Punjab National Bank
SEARCH

IFSC Code Finder Portal Home Admin

Result of "Punjab National Bank" Bank Detail

S.No.	Bank Name	State	City	Branch	IFSC Code	MICR Code	Address	Zip Code	Contact Number
1	Punjab National Bank (PNB)	Uttar Pradesh	Meerut	kjhkjhkjhkj	PUNB12456	PUNB12456	hkjhrehrkh4t3a	123456	9789789798

IFSC Code Finder Portal Home Admin

Result of "kjhkjhkjhkj" Bank Detail

S.No.	Bank Name	State	City	Branch	IFSC Code	MICR Code	Address	Zip Code	Contact Number
1	Punjab National Bank (PNB)	Uttar Pradesh	Meerut	kjhkjhkjhkj	PUNB12456	PUNB12456	hkjhrehrkh4t3a	123456	9789789798